NAVIS: Concurrent Search and Update with Low Position-Seeking Overhead in On-SSD Graph-Based Vector Search

Jaeyong Song*
Seoul National University
jaeyong.song@snu.ac.kr

Hongsun Jang
Seoul National University
hongsun.jang@snu.ac.kr

Changmin Shin
Seoul National University
scm8432@snu.ac.kr

Seongyeon Park
Seoul National University
syeonp@snu.ac.kr

Yong Jae Ryoo
Seoul National University
jae8259@snu.ac.kr

Seo Jin Park
University of Southern California
seojin.park@usc.edu

Jinho Lee
Seoul National University
leejinho@snu.ac.kr

ABSTRACT

On-disk graph-based vector search (GVS) has become the dominant approach for serving large-scale vector databases at high recall. Yet prior systems struggle to sustain concurrent search and update throughput on high-dimensional workloads. We identify the main bottleneck as *position seeking*, the graph traversal that finds neighbors during each update. Unlike search, which returns only the top-K closest vectors, it explores a much larger set to build an R-neighbor list of close neighbors and long-range shortcuts. These neighbors have different distance-accuracy requirements, yet search-oriented packed layouts fetch full vectors with every edgelist access, consuming cache capacity regardless of their role. We present NAVIS, an on-SSD GVS system that redesigns position seeking around this mismatch. At its core, locality-driven decoupling separates vectors from edgelists and packs graph-local edgelists together. This decoupling enables convergence-aware speculative reranking to selectively load full vectors until a sufficient set of close neighbors is resolved, while selecting the remaining shortcuts using approximate distances. It also allows NAVIS to devote cache capacity entirely to edgelists. To make this cache effective, NAVIS keeps the entrance graph fresh via lightweight updates that reuse traversal information already produced during position seeking, thereby maintaining a stable hot set of edgelists near its entry points. Across multiple large-scale benchmarks, NAVIS improves insertion throughput by up to 3.97× and concurrent search throughput by up to 4.00× while reducing search latency by up to 75.69%.

PVLDB Reference Format:
Jaeyong Song, Hongsun Jang, Changmin Shin, Seongyeon Park, Yong Jae Ryoo, Seo Jin Park, and Jinho Lee. NAVIS: Concurrent Search and Update with Low Position-Seeking Overhead in On-SSD Graph-Based Vector Search. PVLDB, 20(3): XXX-XXX, 2026.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/AIS-SNU/NAVIS.

*Work partially done while at University of Southern California.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 3 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Modern applications like retrieval-augmented generation [35], recommendation systems [27, 37], and multimedia search [26, 73] increasingly rely on large-scale vector databases to retrieve semantically similar items at high recall. Among various designs, on-disk *graph-based vector search* (GVS) [20, 21, 48, 53, 54, 61] has emerged as the dominant approach for high-recall search at scale. In on-disk GVS, vectors are vertices in a proximity graph whose edges link nearby points, and a query greedily traverses from a starting vertex toward progressively closer neighbors, loading only the visited portions of the index from SSD. Recent production deployments [12, 48] adopt on-disk GVS since it sustains high recall without holding full vectors in DRAM [18, 36], especially for the 512 to 3072-dimensional vectors that modern encoders produce [5, 45, 46, 50].

In practice, vector databases are continuously updated after deployment. With the rise of agentic systems [24, 31], and as production pipelines [11, 14, 18] now ingest fresh user activity and frequent uploads, on-disk GVS systems are no longer static snapshots. Such updates are handled either by *index rebuild* or by *index update*. As a rebuild often takes several days at large scale [54], many approaches [21, 53, 63, 67] adopt index update to incorporate fresh vectors quickly. However, sustaining high search performance under such concurrent index updates is non-trivial. Despite several proposals for index updates [21, 53, 63, 67], they often fail to sustain high concurrent search and update throughput that modern high-dimensional workloads demand. These systems improve how updates are incorporated or committed, but largely retain the search-oriented traversal used to find neighbors before each update.

We find that ***position seeking*** is the main bottleneck for index update. Position seeking traverses the graph to find neighbors for the newly inserted vertices. This process resembles a search query, but it must explore the graph more extensively to identify a large set of neighbors for graph construction. Specifically, a search query stops once it collects the top-K (e.g., 10) closest vectors, whereas an index update must gather enough neighbors to match the graph's out-degree R (e.g., 64–96) to preserve graph quality. Consequently, position seeking must read far more vectors, leading to costly SSD accesses. Especially on high-dimensional vector databases, position seeking consumes up to 85% of update time and cuts concurrent search throughput by 27.89% on average (Section 3).

A closer look reveals that much of this overhead stems from design choices inherited from search-oriented frameworks. Existing frameworks commonly adopt a *packed layout* [21, 53, 61], which stores each vector together with its edgelist on the same SSD page.

This layout suits search workloads, since each visited vertex typically requires both its vector for distance computation and its edgelist for further traversal. For position seeking, however, we find that this design introduces two key inefficiencies.

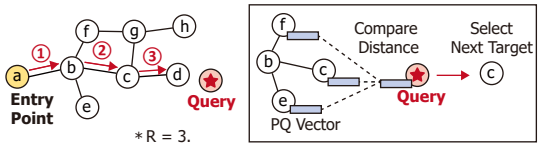
First, search and position seeking impose different distance-accuracy requirements, but the packed layout serves both with the same full-vector I/O. A search query requires accurately distinguishing its top-K results. Instead, position seeking constructs an R-neighbor list whose members serve complementary roles: close neighbors preserve local connectivity, while long-range shortcuts improve graph navigability and reduce traversal I/O [54]. These long-range shortcuts do not require the same degree of distance discrimination as the close neighbors [53]. Nevertheless, accessing their edgelists under the packed layout brings in their full vectors as well, resulting in unnecessary vector reads.

Second, the packed layout exacerbates the cache pressure inherent in position seeking. To identify R neighbors, position seeking explores substantially more vertices than a search query that returns only the top-K, resulting in a much larger working set. This pressure further increases as vector databases grow in both the number and dimensionality of vectors, making it increasingly difficult to retain the working set in memory [66]. One possibility is to selectively cache edgelists, avoiding the management of high-dimensional vectors in cache. However, the packed layout makes such a choice inefficient: caching entire pages wastes cache capacity on vectors, thereby limiting the effective cache size.

From the above observations, we present *NAVIS*, an on-SSD GVS system that redesigns position seeking around *locality-driven decoupling*, which separates vectors from edgelists so a traversal loads edgelists without dragging in vectors and an update writes only the edgelists it changes. Freed of vectors, the new layout packs each page by graph locality, so one read returns several edgelists that a traversal is about to need, at any vector size.

Exploiting this decoupled layout, NAVIS redesigns the vector update system. First, we employ *convergence-aware speculative reranking* to determine when to stop reading the full vectors during position seeking. For each query, it selectively loads full vectors to identify only a sufficient set of close neighbors and fills the remaining neighbors with roughly found shortcuts. This significantly reduces position-seeking overhead through lower I/O.

In addition, locality-driven decoupling enables efficient caching of edgelists in memory, avoiding the cache capacity pressure from the packed layout. However, a compact edgelist cache is effective only when position-seeking traversals exhibit sufficient locality. In GVS systems, such locality is largely shaped by the entrance graph, a sampled (e.g., 1%) in-memory proximity graph that provides good entry points to reduce the traversal hops [20, 61]. Traversals from these entry points tend to reuse common graph paths, creating a compact hot set of edgelists that can be effectively cached. Maintaining a fresh entrance graph is therefore key to preserving this locality. If the entrance graph is omitted or remains unchanged over many insertions [21, 53, 63], the entry points become increasingly distant from newly inserted vertices, dispersing traversal paths and reducing cache effectiveness. NAVIS therefore keeps the entrance graph fresh through a lightweight update that reuses the traversal work already performed during position seeking.



(a) Example of GVS. (b) Procedure of a Single Hop (②).

Figure 1: Overview of graph-based vector search (GVS).

With the above strategies reinforcing one another, NAVIS significantly reduces the overhead of position seeking, thereby boosting the performance of concurrent search and update operations on high-dimensional vector databases. We benchmarked NAVIS on multiple large-scale high-dimensional datasets with state-of-the-art GVS update systems, and it improves insertion throughput by up to 3.97×. Even under such high insertion concurrency, it reduces search latency by up to 75.69% while increasing throughput by up to 4.00×. Even on the low-dimensional dataset, NAVIS improves insertion throughput by up to 3.11× while maintaining the concurrent search performance.

2 BACKGROUND

2.1 Graph-Based Vector Search Systems

Graph-based vector search (GVS) is the dominant index design for high-recall vector retrieval over large-scale vector datasets [15, 20, 21, 40, 53, 54]. GVS organizes high-dimensional vectors into a *proximity graph* [40], a sparse structure in which each vertex stores a vector and each edge links two vectors that are close under a distance metric such as L2. A hyperparameter R caps the per-vertex out-degree and trades exploration breadth for I/O and computation. Existing frameworks typically set R between 64 and 96 and fill each vertex up to it to preserve graph quality.

Fig. 1(a) illustrates a search from an *entry point* (vertex a) toward the closest vertex d. At each hop, the query compares the distances of the current vertex's neighbors and greedily advances to the closest one (e.g., in Fig. 1(b), vertex b selects c), converging within a handful of hops. Because exact distances over full-precision vectors are costly, GVS uses product-quantized (PQ) vectors [30] during traversal and defers exact distances to the visited vertices.

2.2 On-Disk Graph-Based Vector Search

In modern vector databases, the proximity graph from Section 2.1 and its accompanying full vectors often far exceed host DRAM. *On-disk GVS systems* [20, 48, 54, 61] therefore offload both to SSD and load them on demand, providing a cost-effective path to high-recall search over millions of high-dimensional vectors [18, 36]. As shown in Fig. 2(a), they keep a small in-memory *entrance graph* and PQ vectors to make traversal cheap, while the full proximity graph and exact vectors are read selectively from the SSD.

Packed storage layout. To minimize the number of storage I/Os, prior frameworks [20, 48, 54, 61] adopt a *packed layout* in which a vector and its edge list reside contiguously within a 4KiB SSD page as a `[Vector] [Degree] [Edge List]` pair (Fig. 2(a)). This packed layout lets the full vector needed by the final exact top-K selection piggyback on the edgelist read, saving a second I/O.

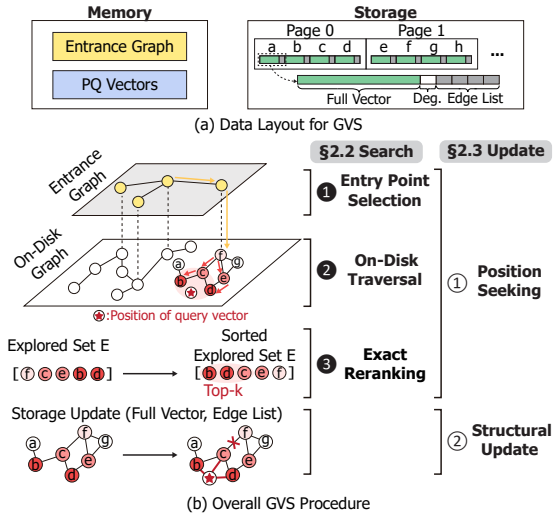


Figure 2: On-disk GVS data layout, search, and update.

For low-dimensional vectors, several pairs share one page, and this page-level locality further reduces the number of storage I/Os [61].

Procedure of search. Using the packed layout, baseline on-disk GVS retrieves the top-K nearest vectors in three stages [20, 54, 61].

❶ *Entry-point selection.* The system first searches the in-memory entrance graph, a sampled subset of the proximity graph with reduced connectivity (e.g., 1% of vertices, R=32) [20, 61]. This process locates well-placed entry points for ❷, reducing on-disk hops.

❷ *On-disk traversal.* From these entry points, a greedy beam search over the on-disk graph advances toward closer neighbors using PQ distances, avoiding heavy computation and I/O overhead. It keeps the current top-$|E_{search}|$ closest vertices in a fixed-size explored candidate pool (`Explored Set`) until no candidate yields a closer neighbor. Although traversal never uses full vectors, each edgelist read under the packed layout brings the full vector along.

❸ *Exact reranking.* Because PQ distance-based examinations are approximate, the system recomputes exact distances for the candidates in E_{search} using their full vectors, which are already loaded into memory via piggybacking. Using these exact distances, the system reranks E_{search} and returns the final top-K vectors. However, many piggybacked vector loads are unnecessary during position seeking. We will discuss this issue in Section 3.

2.3 In-Place Update in On-Disk GVS Systems

Production deployments also receive continuous updates [11, 14, 18, 24, 31]. Among prevailing update-support strategies, we focus on in-place updates [21, 63], which commit each insertion directly on disk to avoid the search-latency fluctuations of buffered merges [53, 67] and quickly reflect fresh vectors into the index. Fig. 2(b) inserts vertex ⋆ into a graph of a–g in two steps, where ① position seeking finds the neighbors of ⋆ and ② structural update wires ⋆ to them.

① *Position seeking.* This step finds neighbors that are semantically similar to the new vector to preserve graph quality, and it runs the same procedure as GVS search (❶–❸). For instance, b, c, and d are closest to ⋆ within the explored set (E_{pos}) and become its neighbors. However, position seeking needs a much larger explored set ($|E_{pos}| \gg |E_{search}|$), because it must gather enough candidates to fill the new vertex up to out-degree R (usually 64–96) rather than only the top-K (e.g., 10). This leads to much heavier storage I/O than a typical search and interferes with concurrent searches.

② *Structural update.* With the neighbors from ① in hand, the system connects the new vertex to them (e.g., ⋆ to b, c, d), prunes an existing edge (e.g., with e, dotted line) when a neighbor would exceed out-degree R, and flushes the modified pages to SSD.

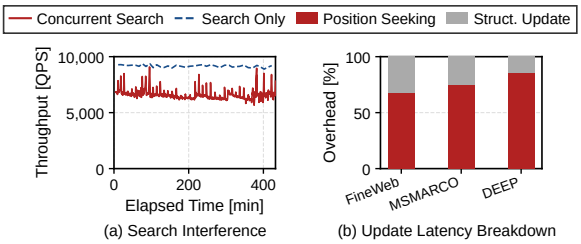


Figure 3: Search interference and update-latency breakdown with OdinANN [21].

3 MOTIVATIONAL ANALYSES

3.1 Position Seeking: Index Update Bottleneck

We first measure how index updates affect concurrent search. Fig. 3(a) compares search-only throughput with search throughput while the state-of-the-art framework (OdinANN [21]) inserts vectors into `MSMARCO` under the default setup in Section 9.1. Search throughput drops by 27.89% on average during the update. Fig. 3(b) separates update latency into position seeking and structural update on all three datasets. Position seeking consumes up to 85% of update time, making it the main bottleneck in search-update interference, and the following subsections identify its root causes.

3.2 I/O Amplification from the Packed Layout

Position seeking constructs an R-neighbor list that contains close neighbors for local connectivity and long-range shortcuts for graph navigability [53, 54]. The close neighbors must be distinguished accurately, while the shortcuts do not require the same distance accuracy. Nevertheless, the packed layout reads every visited vertex's full vector because its edgelist cannot be read separately.

Fig. 4(a) measures this extra I/O on `MSMARCO` with $R = 96$ while varying $|E_{pos}|$ from 64 to 128. We divide the bytes read and written per insertion into `useful vector`, `wasted vector`, `edgelist`, and page-alignment `padding`. Useful and wasted vectors are separated using NAVIS's PQ-distance-based classifier, which we describe in Section 5.2 and validate in Section 9.4. Once $|E_{pos}|$ exceeds R, the amount of `wasted vector` data increases with the explored set. It reaches 44.34% of all bytes read per insertion and consumes SSD bandwidth that could otherwise serve concurrent searches. Additionally, packing loses its page-locality benefit for vectors larger than 2048 bytes because one vector and its edgelist leave no room for another record in a 4 KiB page.

Moreover, during structural update, the packed layout unnecessarily rewrites full vectors. As illustrated in Fig. 4(b), inserting a

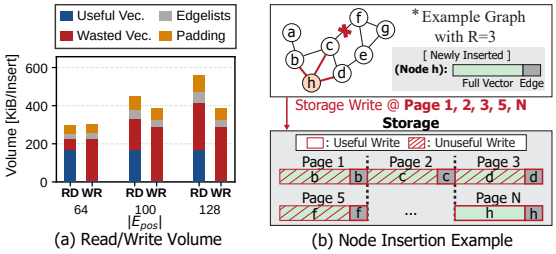


Figure 4: Read and write amplification from packed layout.

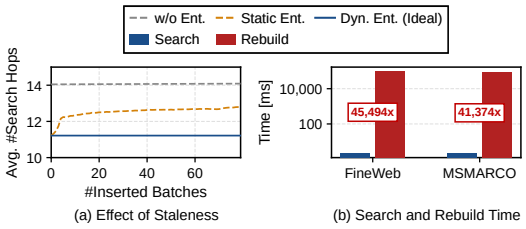


Figure 5: Staleness and rebuild cost of entrance graph.

vertex updates the edgelists of its R neighbors, the pruned neighbor, and the new vertex itself, and each page update rewrites a full vector even though it is unchanged. These rewrites account for up to 74.23% of the bytes written per insertion (Fig. 4(a)).

3.3 Cache Pressure from the Packed Layout and Static Entrance Graph

Limited cache capacity. Position seeking explores far more vertices than top-K search and creates a much larger edgelist working set, requiring careful cache management of reusable edgelists. The packed layout significantly reduces the effective cache capacity because caching an edgelist also brings its co-located full vector into memory. On high-dimensional datasets, these vectors consume most of the cached pages even though the cache aims to reuse edgelists. Thus, the same memory budget holds fewer edgelists and covers a smaller fraction of the position-seeking working set.

Locality loss from a static entrance graph. Before traversing the on-disk graph, the system searches the in-memory entrance graph to select its entry vertices. A fresh entrance graph provides entry vertices that are close to the relevant regions of the current on-disk graph, which keeps the following traversal short. Different traversals from these entry vertices often share their first few hops. The edgelists on those common paths are accessed frequently and form the working set that an edgelist cache can retain.

Existing update-supported systems either omit the entrance graph or leave it unchanged during insertions to avoid its maintenance cost [21, 53]. As new vertices are added, the fixed entry vertices become less effective at reaching the updated regions of the graph. Fig. 5(a) inserts 0.8M vectors into a 0.5M-vector `ImageNet` index, where the static entrance graph requires more search hops as insertion proceeds while a dynamically refreshed one keeps traversal short. Each additional hop accesses another edgelist and expands the working set that the cache must cover. Rebuilding the entire entrance graph is not a practical solution, since one rebuild takes over 40,000× longer than a search (Fig. 5(b)) [54].

Decoupling or caching alone is insufficient. Separating vectors from edgelists lets the cache hold more edgelists, but removes the piggybacked vector read and can add SSD requests during exact reranking. The separated layout therefore needs a policy that reads only the vectors required to select close neighbors. A larger edgelist cache also provides little benefit when a static entrance graph spreads traversal over an expanding working set. An effective solution must therefore coordinate the layout, vector-loading policy, entrance graph maintenance, and edgelist cache.

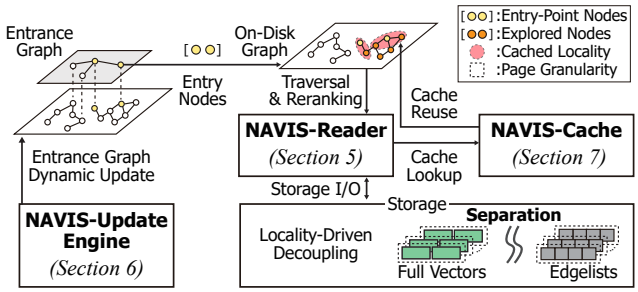


Figure 6: Overview of NAVIS.

4 NAVIS OVERVIEW

As shown in Section 3, position seeking is now the main bottleneck during index update. The packed layout amplifies read and write traffic and wastes cache capacity. A static entrance graph lengthens on-disk traversal and expands the cache working set. NAVIS (Fig. 6) addresses these problems with three components working together.

NAVIS-Reader (Section 5). Locality-driven decoupling stores full vectors and edgelists in separate on-disk files. Traversal can thus read edgelists without fetching full vectors, while structural update writes only the changed records. On this layout, convergence-aware speculative reranking selectively loads full vectors for close neighbors and fills the remaining edge slots with PQ-ranked shortcuts.

NAVIS-Update (Section 6). NAVIS keeps the entrance graph fresh by reusing the candidate sets that position seeking has already produced during insertions. With that freshness, it keeps entry vertices close to newly inserted regions, shortening on-disk traversal and increasing the reuse of edgelists near those entry vertices.

NAVIS-Cache (Section 7). The decoupled layout lets the cache devote memory to edgelists rather than co-located full vectors. Our entrance graph keeps traversal paths short and concentrates accesses near the entry vertices. NAVIS-cache exploits the freed capacity and concentrated access locality by retaining edgelists from frequently accessed paths, while a small admission window filters out edgelists seen only during one-off exploration.

5 NAVIS-READER: LAYOUT-SUPPORTED SELECTIVE VECTOR READING

NAVIS-reader addresses the packed-layout limitations identified in Section 3. It combines locality-driven decoupling, which separates full vectors from edgelists, with convergence-aware speculative reranking, which limits the additional vector requests introduced

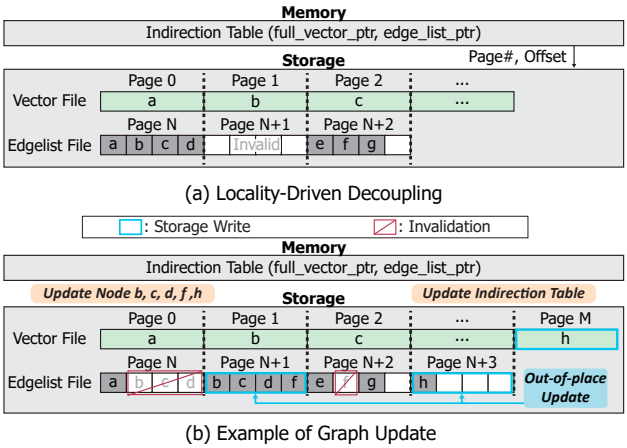


Figure 7: Overview of locality-driven decoupling.

by separation. Together, they reduce position-seeking I/O while retaining page-level locality.

5.1 Locality-Driven Decoupling

To address the packed-layout inefficiencies, we propose *locality-driven decoupling*, which separates full vectors from edgelists while reviving two optimizations originally designed for low-dimensional vectors. For reads, it packs multiple edgelists into each 4 KiB page to exploit page-level locality [61]. For writes, it updates modified edgelists out of place [21] without rewriting unchanged vectors. Fig. 7(a) shows its three data structures. The on-disk *vector file* contains full-precision vectors, while the on-disk *edgelist file* packs multiple edgelists into each page. The memory-resident *indirection table* locates these records by vertex ID.

The edgelist file in locality-driven decoupling enables page-level locality without being affected by the vector dimension because the decoupling lets edgelists gather densely without co-located vectors. In Fig. 7(a), multiple edgelists share a single page regardless of vector size. The nearby vertices' (a, b, c, and d) edgelists reside on one page, so reading a's edgelist during traversal fetches b, c, and d in the same I/O, reducing the total traversal I/O count.

Locality-driven decoupling also enables updates with a much smaller write volume and fewer page writes. As the vectors are decoupled from the edgelists, an update writes only the target vector to the vector file and skips the neighbors' vectors entirely. In Fig. 7(b), only h's own vector is written to the vector file, since unchanged neighbors' vectors are never touched. Locality-driven decoupling also writes only the modified edgelists out of place, so inserting h gathers the edgelists of b, c, d, and f into a single new page. Their old edgelist entries are invalidated, and the indirection table is updated to point to the new locations[1]. This reduces five full-page writes to two edgelist-page writes and one vector-file write, decoupling vector write volume from the out-degree R.

One concern is that page-level locality could be diminished during insertion. Fortunately, the out-of-place write also preserves page-level locality through insertions. Co-updated vertices tend to be adjacent in the graph and are likely to be co-traversed in future queries, so gathering them on a single new page maintains locality without explicit reorganization. For the initial page-locality-aware placement, we utilize the same greedy placement as [61].

The indirection table costs host memory that grows only with the vertex count. It keeps (vertex ID⟶ edge location) and (edge location ⟶ vertex ID) mappings per vertex (two mappings × #vertices × 4 Bytes), while vector locations are computed from the vertex ID rather than stored, which comes to 800MB at 100M vectors and 8GB at 1B. The term is independent of vector dimension and out-degree, so it does not grow on the high-dimensional workloads that make position seeking expensive, and it matches what in-place baselines [21] already pay for out-of-place writes.

[1]The out-of-place update prefers writing to fully invalidated pages if possible, reducing overprovisioned storage. The new page for b, c, d, f reuses an invalidated page.

Algorithm 1 Convergence-Aware Speculative Reranking (CASR).

Input: Query q, candidate set sorted by PQ distance E_{pos}^{sorted}, top-K size K, vector loading group size s (calibrated at warm-up).
Output: Dict with (vertex_id, exact distance) pairs (D), Top-K neighbors of q (T).

$D, T \leftarrow \emptyset, \emptyset$ ▷ Init exact distance set and top-K set
$idx \leftarrow 0$ ▷ Offset in E_{pos}^{sorted} for vector loading
VecLoad(E_{pos}^{sorted}, idx, s); $idx += s$ ▷ Load pipeline start
while PrevVecLoad is non-empty **do**
  WaitPrevVecLoad()
  if $idx < |E_{pos}^{sorted}|$ **then** ▷ Speculative next I/O
    $s \leftarrow \min(s, |E_{pos}^{sorted}| - idx)$
    VecLoad(E_{pos}^{sorted}, idx, s); $idx += s$
  end if
  for all (vertex_id, vector) ∈ PrevVecLoad **do**
    $d \leftarrow$ L2Distance(q, vector)
    $D \leftarrow D \cup \{(\text{vertex_id}, d)\}$ ▷ Exact distance compute/store
  end for
  $T^{new} \leftarrow$ TopK(D, K) ▷ Get new top-K set from the updated D
  if Equal(T, T^{new}) **then break** ▷ Convergence test
  else $T \leftarrow T^{new}$ **endif**
end while
return (D, T)

5.2 Convergence-Aware Speculative Reranking

Realizing vector I/O savings on the decoupled layout requires *convergence-aware speculative reranking*, which selectively loads vectors based on two key insights. First, position seeking has a different goal than search. Search must rerank a small explored set ($|E_{search}|$, typically 10–80) tightly to surface the exact top-K, so every candidate's exact distance matters. Position seeking instead needs a much larger pool ($|E_{pos}| \gg |E_{search}|$) for thorough exploration, but only to surface adequate top-R neighbors for the new vertex, following Vamana's [54] philosophy of combining close neighbors with long-range shortcuts. Exhaustive reranking is therefore unnecessary. Second, PQ distances computed during traversal are a strong signal for close neighbors, since candidates with the smallest PQ distances are most likely the closest under exact distances, which gives a reliable order for issuing full-vector loads.

Convergence-aware speculative reranking replaces the final reranking of position seeking, which conventionally loads full vectors for every candidate in E_{pos}. Under typical settings ($|E_{pos}| = 100$, $R = 96$), nearly the entire pool becomes edges of the new vertex, yet

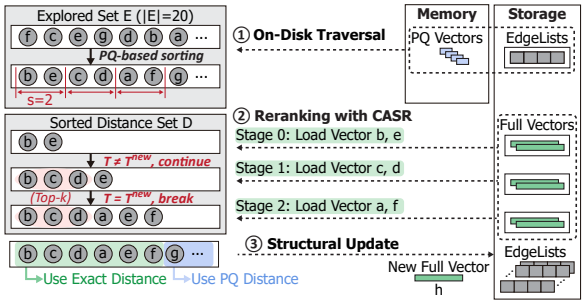


Figure 8: Detailed example of NAVIS-reader.

only close neighbors require exact ranking. We therefore load full vectors in PQ-distance order and track the exact top-K among them, where K is the search's top-K size (e.g., $K = 10$). Once the top-K stops changing, the close-neighbor portion is settled and the remaining R edges fall to shortcut slots, so the number of exact-distance computations adapts to when each query's top-K stabilizes.

Algorithm 1 implements this procedure with low I/O submission overhead, since issuing one I/O at a time per neighbor would be too costly. It loads vectors in groups of size s and pipelines each group's I/O submission with the exact-distance computation of the previous group, hiding submission latency through speculative loads (lines 6–9). After each group, it updates the top-K and compares it with the previous one (lines 10–16), terminating once they are identical and returning the exact-distance set D and the converged top-K.

For position seeking, D provides exact distances for the close-neighbor portion, and the new vertex's remaining R edges are filled with other neighbors based on PQ distances (Section 5.3), since shortcut slots only require diversity rather than exact closeness. Convergence-aware speculative reranking also runs for search by substituting E_{pos}^{sorted} with E_{search}^{sorted} in Algorithm 1, with the top-K set T used directly as the result. The vector I/O reduction gain is smaller than for position seeking since the smaller $|E_{search}|$ leaves less room for early stopping, consistent with the sub-saturation regime ($|E_{pos}| = 64$) in Fig. 4(a). We use different s for the search and position seeking paths because $|E_{pos}| \gg |E_{search}|$.

Group size s is calibrated once at warm-up using 100 queries. To obtain s, we initially set s=1 to get the distribution of the number of vectors needed for the top-K to stabilize across all queries. We then set s to the P25 of the distribution, balancing convergence-detection granularity against I/O submission overhead. We find P25 to be a good sweet spot, as extreme values are clearly suboptimal since s=$|E_{pos}|$ degenerates to a full fetch of $|E_{pos}|$ vectors and very small s submits I/O too frequently, as shown in Section 9.8.

5.3 Update Example

Fig. 8 illustrates an update with NAVIS-reader on the example graph of earlier figures, omitting entry-point selection for brevity. ① **On-disk traversal.** Inserting a new vector traverses the on-disk graph and adds the visited vertices to E_{pos} ($|E_{pos}| = 20$ here, much larger in practice), reading only edgelist pages and comparing distances with host-memory PQ vectors, so locality-driven decoupling avoids full-vector I/O. ② **Exact reranking with convergence-aware speculative reranking.** Sorting E_{pos} by PQ distance gives

Algorithm 2 NAVIS-Update

Require: Query vector q, entrance graph G_{ent}, entrance graph explored set E_{ent}, on-disk explored set during position seeking E_{pos}, subgraph ratio r_{ent}

1: **if** $|G_{\text{ent}}|/|G| < r_{\text{ent}}$ **then** ▷ G_{ent} needs more coverage
2: $E_{\text{inter}} \leftarrow E_{pos} \cap G_{\text{ent}}$ ▷ Select nbr candidates only from G_{ent}
3: $\mathcal{N}_{\text{ent}} \leftarrow \begin{cases} E_{\text{inter}} \cup E_{\text{ent}}[1 : R - |E_{\text{inter}}|], & |E_{\text{inter}}| < R, \\ E_{\text{inter}}[1 : R], & |E_{\text{inter}}| \geq R \end{cases}$
4: q.nbr $\leftarrow \mathcal{N}_{\text{ent}}$
5: **for** $p \in q.nbr : p.\hat{nbr} \leftarrow \text{prune}(p.nbr \cup q)$
6: Lock(G_{ent})
7: $G_{\text{ent}} \leftarrow G_{\text{ent}} \cup q$
8: **for** $p \in q.nbr : p.nbr \leftarrow p.\hat{nbr}$
9: Unlock(G_{ent})
10: **end if**

$E_{pos}^{sorted} = \{\texttt{b},\texttt{e},\texttt{c},\texttt{d},\texttt{a},\texttt{f},\texttt{g},\ldots\}$. With $s = 2$ and $K = 3$, full vectors are loaded group by group, overlapping each group's I/O with the previous group's exact-distance computation, while the running top-K is maintained. The top-K set $\{\texttt{b},\texttt{c},\texttt{d}\}$ stabilizes after Stage 2, so convergence-aware speculative reranking stops after 6 full-vector loads instead of 20, and the remaining $R - 6$ neighbors fall to shortcut slots filled using deflated PQ vectors. ③ **Structural update.** With the neighbor list assembled, the system writes only the new vertex's full vector and the modified edgelist pages, benefiting from locality-driven decoupling's decoupled writes.

6 NAVIS-UPDATE: DYNAMICALLY UPDATING THE ENTRANCE GRAPH

NAVIS-update keeps the entrance graph fresh as the on-disk graph grows. As insertions accumulate, the entry vertices from a static entrance graph become less effective, lengthening position seeking and expanding the edgelist working set. Keeping the entrance graph fresh therefore shortens traversal and reduces the working set that NAVIS-cache must cover. Rebuilding the entire entrance graph is prohibitively expensive, as shown in Section 3.3.

Incrementally adding a vector to the entrance graph presents its own challenge because finding its neighbors normally requires another graph traversal. NAVIS-update avoids this traversal by reusing work that the same insertion has already performed. Entry-point selection produces the entrance graph explored set E_{ent}, while on-disk position seeking produces the explored set E_{pos}. After the on-disk insertion completes, NAVIS-update uses both sets to construct the new vector's entrance graph neighbor list.

Algorithm 2 triggers an update when the entrance graph size $|G_{ent}|$ falls below a threshold ratio of the on-disk graph size ($r_{ent} \times |G|$), with $r_{ent} = 0.01$ following prior work [20, 61] (line 1). It first intersects E_{pos} with G_{ent} to obtain E_{inter}, which retains only vertices already in the entrance graph so that no vertex other than the new vector q is added to G_{ent} (line 2). It then builds q's neighbor list from E_{inter} and E_{ent}, capped at R_{ent} entries and prioritizing E_{inter} because its vertices are typically closer to the query (lines 3–4). Finally, q's neighbors add reciprocal links to q, optionally pruning their lists to the maximum degree R_{ent} (line 5).

Despite performing updates on the in-memory entrance graph, NAVIS-update incurs minimal lock overhead during insertion (lines

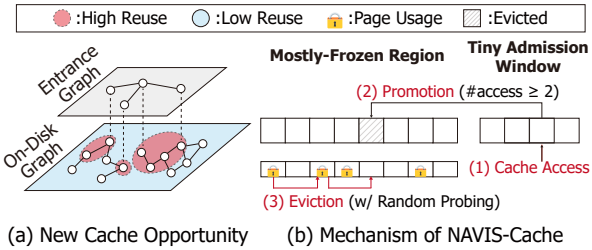


Figure 9: Overview of NAVIS-cache.

6–9). In practice, adding a single vertex with a handful of neighbors into G_{ent} takes only microseconds, and only a small fraction of queries are promoted into the entrance graph. Consequently, lock contention on G_{ent} is negligible even with tens of concurrent insertion threads, and NAVIS-update introduces less than 1% throughput overhead, as shown in Section 9.3.

7 NAVIS-CACHE: PRIORITIZED IN-MEMORY CACHING OF THE PROXIMITY GRAPH

The packed layout wastes cache capacity on co-located vectors.[2] Locality-driven decoupling removes these vector bytes, allowing the same memory budget to retain more edgelist pages. Meanwhile, NAVIS-update keeps entry vertices fresh, shortening position-seeking paths and increasing edgelist reuse near those vertices. Fig. 9(a) illustrates the resulting caching opportunity from them.

A conventional LRU cache is still a poor fit for position seeking. Its long traversals access many edgelist pages only once or a few times. Admitting these low-reuse pages can evict frequently reused pages and lower the hit rate, as confirmed in Section 9.7. Moreover, LRU updates recency state on every hit, adding bookkeeping and synchronization overhead under concurrent access [49].

To exploit entry point locality while avoiding LRU's pitfalls, we design *NAVIS-cache*. Fig. 9(b) illustrates its structure. Inspired by frozen-cache designs [13, 49], NAVIS-cache is partitioned into a *mostly-frozen region* (90%) that preserves the hot region near entry points and protects it from eviction, and a *tiny admission window* (10%) that provides lightweight, usage-aware admission control.

An edgelist must be accessed at least twice within the admission window to be promoted into the mostly-frozen region. The window itself is managed by LRU because of its substantially lower overhead compared to frequency-based policies such as LFU. This rule effectively keeps hot edgelists near entry points while filtering out one-off accesses from long exploration paths, outperforming a full LRU cache as shown in the evaluation (Section 9.7). As NAVIS-update refreshes the entrance graph during insertions, the admission window naturally incorporates newly relevant edgelists.

For the mostly-frozen region, NAVIS-cache uses randomized eviction, selecting a random candidate and, if that entry is currently in use, performing a small number of random probes (up to eight by default) to find an alternative. This avoids expensive tracking structures while preventing evictions of actively used entries.

[2]Although an edgelist cache could conceptually operate at sub-page granularity, each update requires a page-wise read-modify-write to maintain storage consistency. We therefore manage the cache at page granularity.

8 IMPLEMENTATION

I/O Path Optimization. NAVIS issues all storage I/O through `io_uring` [1], which batches submissions and completions on per-thread rings and eliminates the per-request system-call overhead of synchronous read/write paths. Because naïve use still incurs per-call kernel overhead under the large I/O bursts of position seeking, each worker thread owns a private ring of depth 256 and pre-registers both the edge and the vector files via `io_uring_register_files`, so every submission carries an `IOSQE_FIXED_FILE` index instead of a raw file descriptor. Each group of s vector reads is committed in a single `io_uring_submit_and_wait` call and its completions are harvested in bulk via `io_uring_peek_batch_cqe`, paying a single enter-syscall per group and keeping this overhead near-constant.

NAVIS-Cache Control Details. Concurrent search and update force NAVIS-cache to coexist with two write-side mechanisms, a read-modify-write (RMW) page cache that holds dirty pages until they are flushed, and locality-driven decoupling's out-of-place edge updates that relocate edgelists across pages.

NAVIS-cache is a write-through cache for clean read-side traffic, while only the RMW page cache holds dirty pages, so eviction from NAVIS-cache never has to commit data and the RMW path never has to negotiate freshness with NAVIS-cache. On a search or position-seeking miss, the worker probes the RMW cache first to pick up any in-flight modifications, falls through to NAVIS-cache for a clean hot page, and only issues storage I/O on a double miss, after which the page is admitted to the RMW cache and then to NAVIS-cache's admission window. Structural update operates exclusively on the RMW cache, pinning all dirty pages with reference counts and releasing the page lock only after the page is written to storage and any matching NAVIS-cache entry is refreshed in place, an ordering that ensures NAVIS-cache never holds a page newer than disk.

The out-of-place edge updates in locality-driven decoupling add a second consistency obligation, since an edgelist may move to a freshly allocated edge page, rendering the old slot stale. We enforce freshness through the indirection table rather than through cache metadata. A reader resolves `id` → `edge_list_ptr` from the table before consulting any cache, so a hit on a relocated id targets the new page and the stale page is never queried again. When all slots in a 4 KiB edge page have been invalidated, the indirection layer issues an eviction hint to NAVIS-cache for that page number, so the page is reclaimed before the cold-path randomized eviction would touch it. Per-vertex tombstones and version counters are unnecessary, since invalidation is page-grained and the indirection table already provides the freshness guarantee. This keeps NAVIS-cache lock-free on the read path and free of dirty-page state management.

9 EVALUATION

9.1 Experimental Setup

Hardware. We used a single server for all evaluations, with an AMD Ryzen 9 7950X3D CPU (16C 32T, 4.2–5.7GHz), 128GB of DDR5 memory (4× DDR5 5600MT/s), one Micron Crucial PCIe 5.0 T705 4TB SSD, and Ubuntu 24.04 LTS as the operating system.

Datasets. We evaluated NAVIS on four datasets (Table 1) and conducted the main experiments on three that span different dimensionalities and database scales. `FineWeb` documents are embedded with EmbeddingGemma [58] and `ImageNet` images with CLIP [50],

Table 1: Dataset Configurations Used in NAVIS Experiments

Dataset	Dim. (float)	#Vectors	Type	$\|E_{search}\|$	$\|E_{pos}\|$	PQ Bytes
FineWeb [39]	768	60,000,000	Text	40	100	128
MSMARCO [3]	768	100,000,000	Text	40	100	128
DEEP [2]	96	120,000,000	Image	40	200	32
ImageNet [34]	512	1,300,000	Image	40	128	64

while DEEP uses low-dimensional vectors generated by compression [2] to show that NAVIS also performs well on low-dimensional datasets. ImageNet is used only to evaluate a production baseline (Qdrant [48]) and a clustering-based baseline (SPFresh [65]) because neither reaches a comparable operating point on the high-dimensional, large-scale vector datasets of the main study within a practical time budget. We inserted vectors equivalent to 20%, 100%, 20%, and 30% of the base index size for FineWeb, MSMARCO, DEEP, and ImageNet, respectively. All datasets use $R = 96$ following prior work [21, 54], and the image-based vectors of DEEP and ImageNet require much larger $|E_{pos}|$ values of 200 and 128 because they are highly similar to one another. Following standard methodology [21, 54], we randomly selected 10,000 queries per dataset and targeted Recall@10 of 0.95, 0.95, 0.85, and 0.95 for the four datasets, respectively, with $|E_{search}| = 40$ for all, using a lower target for DEEP because it has fewer dimensions than the others.

Baselines. We compared NAVIS against four main GVS baselines and two additional baselines, for six baselines in total:

- **FreshDiskANN** [53]: a buffered-insertion design using a host-side buffer that is flushed and merged into the on-disk graph once the buffer reaches 6% of the base graph size.
- **IP-DiskANN** [63]: a concurrent insertion baseline implemented in the current DiskANN3 library [41]. It performs in-place updates to prevent search fluctuation.
- **OdinANN** [21]: a state-of-the-art in-place update that improves IP-DiskANN via better lock concurrency control, neighbor pruning during structural update, and overprovisioned writes.
- **OdinANN+$**: OdinANN with NAVIS-cache. Since OdinANN retains the packed layout, NAVIS-cache must cache packed pages (vectors co-resident with edgelists), so its effective edgelist cache size is smaller than NAVIS's for the same memory budget.
- **NAVIS (Ours)**: a high-performance concurrent insertion framework that reduces unnecessary vector reads and employs a dynamic entrance graph with an efficient cache.

All systems used a beamwidth of four. Since FreshDiskANN employs buffered insertion and merges the buffered graph whenever the index grows by 6% of the base graph size, we derived its insertion throughput by averaging it over time. We tested only FreshDiskANN as the representative baseline for buffered-insertion approaches [53, 67], since these approaches are well known for concurrent search fluctuations under large updates [21], which our evaluations also confirm. Prior work [20, 61] that does not support concurrent updates is excluded. To test the limitations of current production and cluster-based systems, we also benchmarked **Qdrant** [48] and **SPFresh** [65]. We report them only for ImageNet in Section 9.8, since neither reached a comparable operating point on the main datasets within a practical evaluation time.

Other settings. We configured 22 search threads and 10 insertion threads by default, following prior work [21]. Overall performance evaluations report up to 12 hours of insertions on slow baselines (i.e., FreshDiskANN and IP-DiskANN), and other experiments report the first 100K insertions. For search-only tasks, we warm up cache-based baselines by running the full 10,000-query search ten times. OdinANN+$ and NAVIS use a 16GB host-memory cache for a fair comparison, matching the memory usage of FreshDiskANN, whose insertion buffer takes much more host memory. All GVS baselines enable the entrance graph with ten entry points.

9.2 Concurrent Search-Insert Performance

In Fig. 10–12, NAVIS delivers substantial improvements over the baselines, showing up to 3.97× higher insertion throughput. Even under such high concurrent insertion, NAVIS achieves better search performance, up to 4.00× higher average queries per second (QPS), and up to 75.69% lower average latency, maintaining stable recall. Its curves terminate earlier because the workload completes faster.

FineWeb. On FineWeb, NAVIS achieves 1.36/3.63/1.60/1.54× higher insertion throughput and 1.17/4.00/1.37/1.19× higher average QPS than FreshDiskANN, IP-DiskANN, OdinANN, and OdinANN+$, respectively, while reducing average latency by 18.98/75.69/25.26/15.06%. The NAVIS-cache helped OdinANN+$ to some degree, but without the locality-driven decoupling, the performance improvement over OdinANN is limited, especially for insertion, due to redundant vector I/O and the small effective cache size from the packed layout. FreshDiskANN exhibits the buffered-merge instability previously reported by [21], where its worst-case search QPS drops 79.1% relative to its average during merge windows, visible as the spikes in Fig. 10(b). IP-DiskANN avoids these fluctuations through in-place updates, but its concurrency control between searches and insertions limits both insertion and search throughput, which OdinANN addresses through improved lock concurrency control. Recall fluctuates for FreshDiskANN before each buffered batch is merged into the on-disk graph and recovers after the merge. NAVIS, OdinANN, and OdinANN+$ converge to similar recall (near 97%), all comfortably above the 95% target.

MSMARCO. On MSMARCO, NAVIS improves insertion throughput by 2.74/2.15/1.63/1.83× and average QPS by 1.06/3.14/1.26/1.20×, while reducing latency by 9.43/69.18/21.76/17.78% over FreshDiskANN, IP-DiskANN, OdinANN, and OdinANN+$, respectively. The search QPS gain over FreshDiskANN is smaller than for FineWeb because NAVIS's insertion throughput on MSMARCO far exceeds FreshDiskANN's, interfering more with concurrent search. Even so, NAVIS provides more stable and better search performance than FreshDiskANN. OdinANN+$ benefits from NAVIS-cache for search, but due to the low hit rate from the small effective size with the packed layout, it shows lower insertion throughput than OdinANN. Regarding recall, FreshDiskANN again shows fluctuation (93%–97%) as in the FineWeb case, and NAVIS (97%) matches FreshDiskANN's max recall while sitting 1% below OdinANN (98%), still comfortably above the 95% target.

DEEP (low-dimensional dataset test). Although NAVIS mainly targets high-dimensional vector databases, it remains competitive

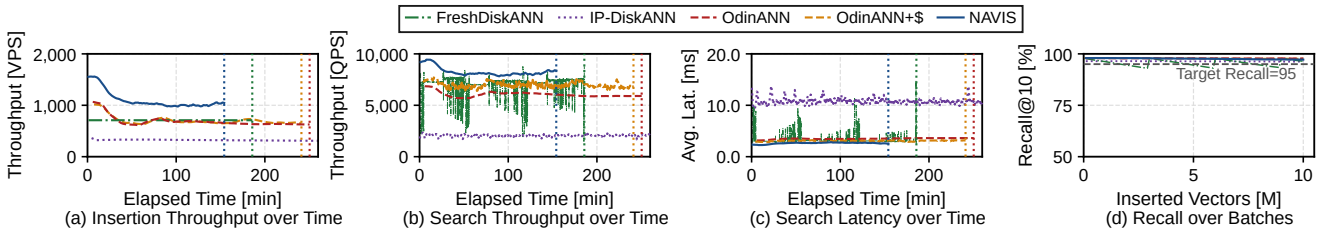


Figure 10: Update throughput and search performance under concurrent updates of baselines and NAVIS on the `FineWeb` dataset.

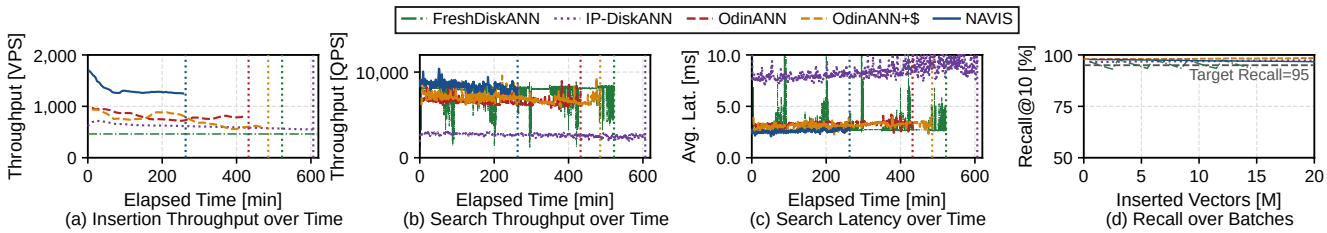


Figure 11: Update throughput and search performance under concurrent updates of baselines and NAVIS on the `MSMARCO` dataset.

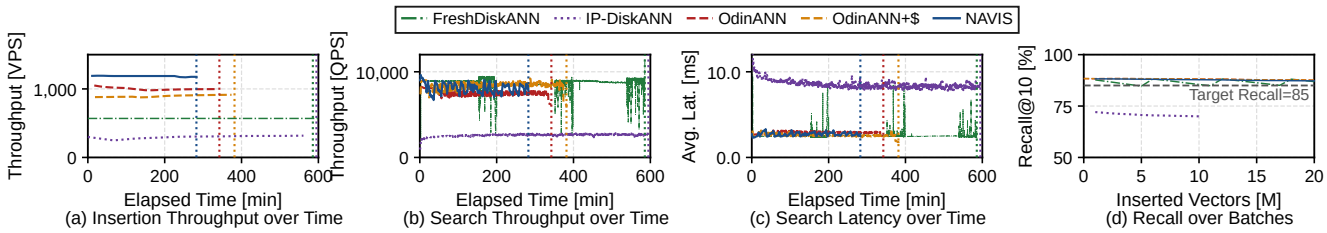


Figure 12: Update throughput and search performance under concurrent updates of baselines and NAVIS on the `DEEP` dataset.

on DEEP. It improves insertion throughput by 2.07/3.96/1.21/1.35× over FreshDiskANN, IP-DiskANN, OdinANN, and OdinANN+$, respectively, while achieving comparable search throughput (0.93/3.11/1.08/0.96× QPS). This advantage stems from NAVIS's reduced vector I/O and an optimized caching strategy that exploits the entrance graph locality. OdinANN+$ confirms this, with higher search but lower insertion throughput than OdinANN since it does not reduce vector I/O. NAVIS matches OdinANN's recall, while FreshDiskANN shows the same merge-induced fluctuation. IP-DiskANN reaches only 69.98% under the same $|E_{search}| = 40$ used by the other systems.

9.3 Insert-Heavy Tests and Time Breakdown

We conducted insert-heavy benchmarks varying the #insert/#search thread ratio on `MSMARCO` (Fig. 13(a) and (b)), and broke down the insert time during the run in Fig. 13(c). In the insert-only test (i.e., 32/0 #insert/#search thread ratio), NAVIS's vector I/O reduction and effective cache yield 2.01/2.12× higher throughput than OdinANN and OdinANN+$, respectively, while substantially reducing insert latency, especially on tail latencies (P90/P99, up to 65.74% over OdinANN). In the insert-heavy tests (Fig. 13(b)) with the insert/search thread split over 10/22, 16/16, and 22/10, insertion throughput (hollow markers) over OdinANN rises from 1.64× at

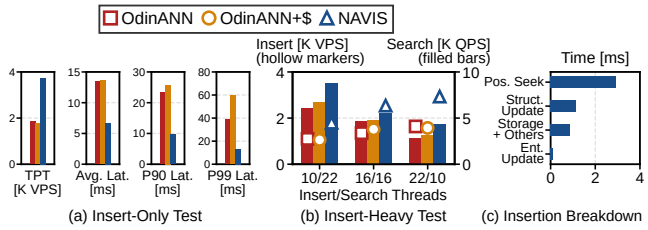


Figure 13: Insert-intensive tests and insert time breakdown.

10/22 to 1.79× at 22/10, since the position-seeking traffic that NAVIS reduces grows with the insert rate. In Fig. 13(c), NAVIS's vector I/O reduction shrinks the position-seeking share relative to Fig. 3 (74.83% → 58.24%), and `Ent. Update`, the entrance graph update overhead of NAVIS-update, takes less than 1% of the total time.

9.4 Ablation Studies

To evaluate the contribution of NAVIS's components, we conduct two complementary ablation studies. Table 2 first isolates convergence-aware speculative reranking (CASR) by comparing full reranking (`Full`) with CASR enabled (`+CASR`) while keeping all other NAVIS components unchanged. To compare the recall

Table 2: Ablation Analysis of CASR

Dataset	Type	Insert TPT	Search TPT	#Vec. Reads	Recall@10
FineWeb	Full	633.16	6016.33	103.31	97.73
	+CASR	1171.57	9541.02	65.14	97.19
MSMARCO	Full	796.40	6705.56	104.30	98.43
	+CASR	1408.89	8432.60	65.48	97.85
DEEP	Full	1002.37	7511.11	202.42	87.56
	+CASR	1181.06	8096.12	89.27	87.45

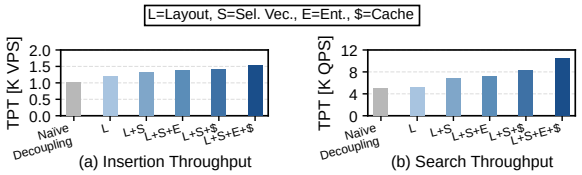


Figure 14: Effect of NAVIS components.

accurately, we did not apply CASR for the concurrent search. CASR reduces vector reads by up to 55.90%, increasing insertion and search throughput by up to 1.85× and 1.59×, respectively, while keeping the recall loss at most 0.58 pp.

Fig. 14 then builds NAVIS incrementally on MSMARCO. **Naïve Decoupling** uses our codebase and I/O path but only separates vectors and edgelists, without locality-aware placement, selective vector reads, entrance graph maintenance, or NAVIS-cache. **Layout (L)** adds locality-aware page placement, improving insertion/search throughput by 1.19/1.04×, and **L+Sel. Vec. (L+S)** adds CASR for a further 1.10/1.32×, showing that decoupling becomes substantially more effective with selective vector reads. Adding NAVIS-update (**+Ent., L+S+E**) or NAVIS-cache (**+$, L+S+$**) separately to L+S improves insertion/search throughput by 1.05/1.05× and 1.07/1.22×, respectively, capturing only part of their joint benefit. Adding NAVIS-update after NAVIS-cache, as in **L+S+E+$** (i.e., NAVIS), provides a further 1.08/1.26× over L+S+$, raising the marginal search benefit of NAVIS-update from 1.05× without NAVIS-cache to 1.26× with it. This larger combined benefit shows that a fresh entrance graph and NAVIS-cache reinforce each other, and the full system reaches an overall 1.51/2.11× insertion/search speedup over Naïve Decoupling.

Consistent with this interaction, Fig. 17(c) separately isolates NAVIS-update with NAVIS-cache. Removing NAVIS-update under the same cache policy lowers the cache hit rate by 6.78 pp, showing that a fresh entrance graph creates the stable locality that NAVIS-cache exploits. Applying the same cache to the packed layout yields only limited gains due to the smaller effective capacity for edgelists. Together, these results show how locality-driven decoupling frees cache capacity, NAVIS-update creates stable locality, and NAVIS-cache converts both into fewer SSD reads.

9.5 Search Tail Latency Analysis

Fig. 15 shows the P90/P99 latency profile on MSMARCO from the experiments in Section 9.2. NAVIS reduces the maximum P90 latency by 64.01%/65.78% relative to FreshDiskANN and OdinANN, and the maximum P99 latency by 92.22%/84.78%, respectively. This improvement primarily stems from NAVIS's reduction in vector I/O, which dominates long-latency outliers under concurrent updates.

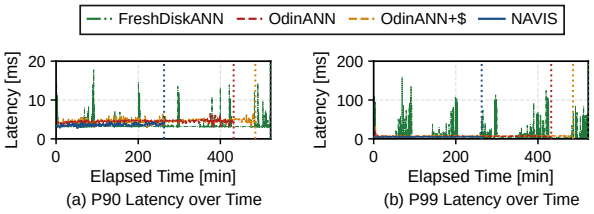


Figure 15: Search tail latency (P90/P99) profiling results.

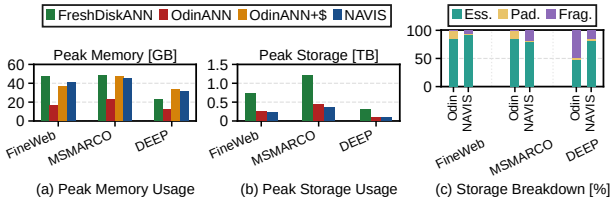


Figure 16: Memory usage and storage usage with breakdown.

9.6 Memory and Storage Usage Analyses

Fig. 16 shows the peak memory consumption and storage usage of NAVIS compared to baselines. FreshDiskANN consumes more host memory than the others on high-dimensional datasets because of its host-memory merge buffer, so we configured OdinANN+$ and NAVIS with a 16GB NAVIS-cache to match its peak, while OdinANN uses the least memory as it has no cache. On DEEP, FreshDiskANN's host memory drops below OdinANN+$ and NAVIS because low-dimensional vectors yield a smaller insertion buffer, while both retain the 16GB NAVIS-cache for consistency across datasets. Regarding storage, FreshDiskANN utilizes significantly more storage because it adopts a double-buffered storage scheme during merge. NAVIS and OdinANN show much lower storage usage because they apply non-buffered updates, unlike FreshDiskANN, with NAVIS slightly below OdinANN because it eliminates per-page padding.

Fig. 16(c) breaks the storage footprint of both systems into essential bytes, padding, and fragmentation from out-of-place writes at the end of a full insert run. NAVIS fragments only the edgelist file, and its allocator reuses fully invalidated pages when placing new writes, so on DEEP its fragmentation stays at 15.52% against OdinANN's 49.46%, which fragments its packed pages. What OdinANN avoids in fragmentation on FineWeb and MSMARCO, it already pays in padding. A page there holds a single vertex, so every rewrite invalidates the whole page and leaves no fragmentation behind, an outcome that follows from the padding those pages already carry. The property does not carry over to DEEP, where a page serves several vertices and fragmentation grows sharply.

9.7 SSD Sensitivity Test, Search-Only Test, and Cache Policy Test

To show that NAVIS consistently provides gains on more common SSDs, Fig. 17(a) repeats the concurrent search-insert test on MSMARCO across a SATA SSD, a PCIe 4.0 NVMe SSD, and the PCIe 5.0 NVMe SSD used elsewhere. On the more common PCIe 4.0 device, the gain over OdinANN persists at 1.71× for search and 1.53× for

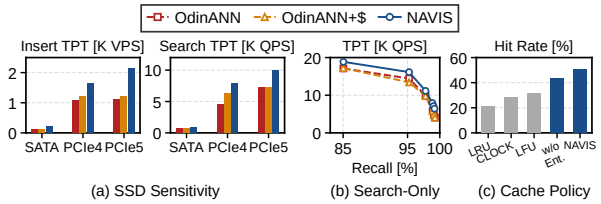


Figure 17: SSD sensitivity, search-only test, and cache policies.

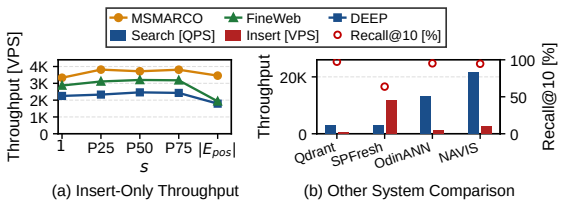


Figure 18: Sensitivity of group size and comparison with production (Qdrant) and cluster-based (SPFresh) baselines.

insertion. SATA limits every system by the device, so we report it as an extreme end point rather than a deployment target.

While NAVIS primarily targets update overhead during position seeking, NAVIS-reader and NAVIS-cache also benefit search-only workloads, as shown in Fig. 17(b) on the `MSMARCO` dataset with 32 search threads and zero insert threads. Across all recall levels with various $|E_{search}|$, NAVIS provides higher QPS than OdinANN and OdinANN+$, up to 1.60×.

Fig. 17(c) tests NAVIS-cache's policy choices on `FineWeb` with the cache forced to 4GB, since with locality-driven decoupling a 16GB cache yields over 90% hit rate across all baselines. We compare well-known LRU, FIFO with second chance (CLOCK), and LFU, and also NAVIS-cache without NAVIS-update (`NAVIS-wo Ent`) to assess how NAVIS-update improves locality near entry points. As expected, conventional policies suffer from cache pollution caused by low-reuse edgelists and fail to retain near-entry-point structures in the cache, whereas NAVIS-cache effectively prevents such pollution and improves the hit rate by 19.28 pp over LFU, and NAVIS-update adds another 6.78 pp over the configuration without it.

9.8 Group Size Sensitivity and Comparison with Other Baselines

Group size s is calibrated per index rather than fixed across workloads, and the mechanism is insensitive to the exact value. As described in Section 5.2, s is set at warm-up from the P25 of the per-query convergence distribution, with separate values for the search and position-seeking paths, so a new dataset needs no manual tuning. In Fig. 18(a), we tested the sensitivity of NAVIS to group size s with the insert-only test using 32 threads on all three main datasets. As expected, $s = 1$ submits I/O too frequently and $s = |E_{pos}|$ loads all vectors in the candidate set, both showing lower insertion throughput than other s choices. The curves are flat across the middle of the range on every dataset, with the P25 to P75 spread within 5.71%, so the P25 rule is a convenience rather than a requirement. We also compared NAVIS against a production baseline (Qdrant [48]) and a clustering-based baseline (SPFresh [65]) on the `ImageNet` dataset in Fig. 18(b). For Qdrant, we used the new HNSW [40] features with storage from `version 1.16.3`, which use a strategy similar to NAVIS and the GVS baselines. Qdrant shows high recall, but its search and insert throughput are both significantly lower. SPFresh achieves high insert throughput from its batched, cluster-based design, but falls short on recall and search throughput. Even on `ImageNet`, each baseline trails the GVS systems (i.e., OdinANN and NAVIS) in at least one of search throughput, insertion throughput, or recall, so we focus the main evaluation on GVS systems.

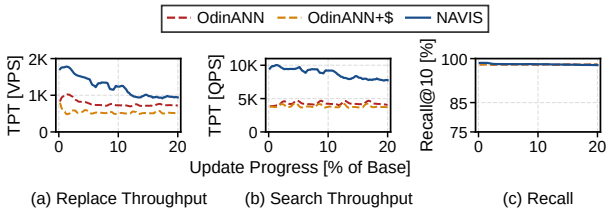


Figure 19: Deletion benchmark by replacing the vectors.

9.9 Deletion Support

NAVIS also supports deletion through the indirection table. A deletion marks the vertex invalid in the table, searches skip invalid candidates using a slightly enlarged E_{search}, and a bulk consolidation runs once the invalidated fraction crosses a threshold, as in OdinANN. Following the benchmark from [21, 53], Fig. 19 runs a vector replacement (i.e., delete → insert) benchmark on `FineWeb`, replacing 20% of vectors. NAVIS sustains 1.58/2.05× higher concurrent update/search throughput than OdinANN across the run, and `Recall@10` stays within 0.75 pp of its starting value for all systems. OdinANN+$ falls below OdinANN under this workload because deletions invalidate cached entries faster than the workload can replace them in a small effective cache size due to the packed layout, resulting in cache management overhead without benefit.

10 DISCUSSION

Deployment conditions. The smaller gain on low-dimensional `DEEP` shows that NAVIS's vector-I/O reduction is most beneficial on high-dimensional datasets, where avoided reads save more SSD traffic (Table 2). The benefit grows with the update rate as more insertions place position-seeking traffic on the shared SSD (Fig. 13(b)). NAVIS remains advantageous across SSD classes, although the gain does not scale monotonically with device bandwidth because computation and synchronization also matter (Fig. 17(a)). NAVIS-cache is most advantageous under limited memory, because locality-driven decoupling enlarges effective cache capacity and our entrance graph stabilizes the hot set. Its advantage over conventional policies narrows once the hot set fits in memory (Fig. 17(c)).

Query-wise vector selection. Insertion queries vary in position-seeking difficulty, requiring different numbers of full-vector reads for their exact top-K to stabilize. NAVIS therefore uses query-wise adaptive vector selection. To test this choice, we replace CASR with fixed-length vector selection used by another decoupled approach [38], leaving the rest of NAVIS unchanged. Table 3

Table 3: Fixed and Query-Adaptive Policy Comparison

Method	#Vec. Reads	Insert TPT	Search TPT	Recall@10
Fixed	30	1538.00	14301.61	92.53
Fixed	65	1156.34	10868.60	93.64
CASR	65.14	1171.57	9541.02	97.19

reports this controlled comparison on `FineWeb`. A fixed budget of 65 closely matches the average number of vector reads under CASR. It achieves comparable insertion throughput and slightly higher concurrent search throughput by fetching all selected vectors in one I/O submission. However, its recall is far below CASR and misses the 95% target, showing that a fixed budget cannot handle the varying convergence behavior of insertion queries.

Application on emerging workloads. Agentic memory is an emerging use case for concurrent vector search and update. Mem0 adds and consolidates memories during serving, while A-Mem links new memories and updates existing representations [9, 64]. These updates must become searchable during retrieval, and they require stable serving/update throughput and latency. When such stores grow beyond DRAM, NAVIS can provide an on-SSD index for this concurrent retrieval and update workload.

11 RELATED WORK

Graph-based vector databases. Graph-based vector search (GVS) spans many platforms [7, 10, 19, 29, 32, 33, 40, 44, 56, 61, 68–70, 72, 74]. On-disk systems [17, 20, 28, 32, 47, 51, 54] provide low-cost scaling by storing the full graph on storage and compressed vectors in memory. Recent systems [21, 38, 53, 63, 71] also support runtime graph updates. FreshDiskANN [53] supports inserts and deletes via buffered updates, causing significant concurrent search performance fluctuations during merge. Thus, IP-DiskANN [63] introduced direct in-place updates, while OdinANN [21] refines this model with approximate concurrency control and write overprovisioning. We directly evaluate both in Section 9.2. CleanANN [71] reduces in-memory graph-update overhead and is validated in memory rather than on disk. It therefore targets a different scenario, and its graph algorithm is complementary to NAVIS.

Decoupled on-disk layouts. Separating vectors from topology is not itself new. DGAI [38] decouples vectors from the topology data and reduces redundant vector I/O during updates, but reranks a *fixed* top-τ candidate set selected at warm-up with enhanced PQ codebooks and keeps its entrance graph static. NAVIS instead selects vectors per query until the exact top-K settles, refreshes the entrance graph, and exploits the resulting locality via NAVIS-cache. Table 3 and Section 10 (Query-wise vector selection) isolate this policy difference within NAVIS, showing that fixed-length selection misses the target recall even when its average vector reads match CASR. DGAI's enhanced PQ compression remains complementary to these selective vector reads. COMPASS [52] (DecoupleVS in the latest preprint) decouples vector data and index metadata for component-specific lossless compression and proposes techniques to maintain comparable search and insertion throughput despite compression overhead. It targets storage efficiency rather than position-seeking I/O, and its compression could be applied to locality-driven decoupling's vector file.

GPU-accelerated GVS. To accelerate graph-based vector search, CAGRA [44] keeps the graph and FP32/FP16 vectors in GPU memory and co-designs graph construction and traversal for GPU parallelism. BANG [57] instead keeps the graph and full vectors in host memory, using GPU-resident PQ codes for traversal followed by exact reranking. Both target static, memory-resident search rather than concurrent updates over an SSD-resident graph. These approaches complement NAVIS since NAVIS supports large, high-dimensional datasets with an SSD-resident graph, while GPUs can accelerate the remaining distance computations after NAVIS reduces position-seeking I/O.

Concurrent search and update systems. Cluster-based methods [4, 6, 8, 12, 16, 23, 25, 30, 43, 55, 65] partition vectors, often with k-means, and localize updates instead of modifying multiple graph edges. Recent dynamic systems use incremental rebalancing in SPFresh [65], temperature-based selective reindexing in Ada-IVF [42], and cost-based partition management in Quake [43]. Production systems [22, 48, 59, 60, 62] also support concurrent search and update, including Qdrant [48], which pairs HNSW [40] with on-disk storage and concurrent updates. Other production systems avoid direct updates to an SSD-resident graph through different architectures. Milvus [22, 60] buffers vectors in a growing in-memory segment and later seals it into an immutable indexed segment, reproducing the merge-window fluctuations of FreshDiskANN. Weaviate [62] and Vespa [59] instead update mutable in-memory HNSW graphs directly, as in-memory libraries [12, 40] do. Section 9.8 evaluates Qdrant and SPFresh. On `ImageNet`, each trails the GVS systems in search throughput, insertion throughput, or recall, and neither reached a comparable operating point on our large high-dimensional datasets within a practical evaluation time.

12 CONCLUSION

This paper presents NAVIS, an on-disk GVS system enabling concurrent, high-performance search and update with low position-seeking overhead. We identify position seeking as the main source of search–update interference, driven by uniform full-vector I/O and cache pressure in packed layouts and entrance graph staleness. NAVIS mitigates these issues because decoupling enables selective vector reads and edgelist-only caching, while entrance graph maintenance preserves a stable hot set. Across benchmarks, NAVIS achieves up to 3.97× higher insertion throughput. It provides up to 4.00× higher search throughput and up to 75.69% lower search latency.

ACKNOWLEDGMENTS

Jinho Lee is the corresponding author. This work was primarily supported by Samsung Electronics. Jinho Lee is additionally funded by National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2026-25495605), and Institute of Information & communications Technology Planning & Evaluation (IITP) (RS-2024-00395134, RS-2024-00347394, RS-2026-25548502, RS-2026-25549926, RS-2021II211343). Part of the infrastructure used in this work was supported by Korea Basic Science Institute (National Research Facilities and Equipment Center) grant funded by the Ministry of Science and ICT (No. RS-2025-00564840).

REFERENCES

[1] Jens Axboe. 2019. Efficient IO with io_uring. Retrieved October 24, 2026 from https://web.archive.org/web/20251229090652/https://kernel.dk/io_uring.pdf

[2] Artem Babenko and Victor S. Lempitsky. 2016. Efficient Indexing of Billion-Scale Datasets of Deep Descriptors. In *Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2055–2063. https://doi.org/10.1109/CVPR.2016.226

[3] Payal Bajaj, Daniel Campos, Nick Craswell, Li Deng, Jianfeng Gao, Xiaodong Liu, Rangan Majumder, Andrew McNamara, Bhaskar Mitra, Tri Nguyen, Mir Rosenberg, Xia Song, Alina Stoica, Saurabh Tiwary, and Tong Wang. 2016. MS MARCO: A Human Generated MAchine Reading COmprehension Dataset. *arXiv preprint arXiv:1611.09268* (2016). https://doi.org/10.48550/arXiv.1611.09268

[4] Dmitry Baranchuk, Artem Babenko, and Yury Malkov. 2018. Revisiting the Inverted Indices for Billion-Scale Approximate Nearest Neighbors. In *Proceedings of the 15th European Conference on Computer Vision (ECCV)*. 209–224. https://doi.org/10.1007/978-3-030-01258-8_13

[5] Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*. 2318–2335. https://doi.org/10.18653/v1/2024.findings-acl.137

[6] Kangqi Chen, Rakesh Nadig, Manos Frouzakis, Nika Mansouri Ghiasi, Yu Liang, Haiyu Mao, Jisung Park, Mohammad Sadrosadati, and Onur Mutlu. 2025. REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing. In *Proceedings of the 52nd International Symposium on Computer Architecture (ISCA)*. 1171–1192. https://doi.org/10.1145/3695053.3731116

[7] Patrick Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference 2023 (WWW)*. 3225–3235. https://doi.org/10.1145/3543507.3583318

[8] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-Efficient Billion-Scale Approximate Nearest Neighborhood Search. In *Proceedings of the Advances in Neural Information Processing Systems 34 (NeurIPS)*. 5199–5212. Retrieved October 24, 2026 from https://proceedings.neurips.cc/paper_files/paper/2021/hash/299dc35e747eb77177d9cea10a802da2-Abstract.html

[9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. In *Proceedings of the 28th European Conference on Artificial Intelligence (ECAI)*. 2993–3000. https://doi.org/10.3233/FAIA251160

[10] Benjamin Coleman, Santiago Segarra, Alexander Smola, and Anshumali Shrivastava. 2022. Graph Reordering for Cache-Efficient Near Neighbor Search. In *Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS)*. 38488–38500. https://doi.org/10.52202/068431-2789

[11] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys)*. 191–198. https://doi.org/10.1145/2959100.2959190

[12] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data (BigData)* 12, 2 (2026), 346–361. https://doi.org/10.1109/TBDATA.2025.3618474

[13] Gil Einziger, Roy Friedman, and Ben Manes. 2017. TinyLFU: A Highly Efficient Cache Admission Policy. *ACM Transactions on Storage (TOS)* 13, 4 (2017), 1–31. https://doi.org/10.1145/3149371

[14] Chantat Eksombatchai, Pranav Jindal, Jerry Zitao Liu, Yuchen Liu, Rahul Sharma, Charles Sugnet, Mark Ulrich, and Jure Leskovec. 2018. Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time. In *Proceedings of the 2018 World Wide Web Conference (WWW)*. 1775–1784. https://doi.org/10.1145/3178876.3186183

[15] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search with the Navigating Spreading-Out Graph. *Proceedings of the VLDB Endowment* 12, 5 (2019), 461–474. https://doi.org/10.14778/3303753.3303754

[16] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2014. Optimized Product Quantization. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 36, 4 (2014), 744–755. https://doi.org/10.1109/TPAMI.2013.240

[17] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023 (WWW)*. 3406–3416. https://doi.org/10.1145/3543507.3583552

[18] Google. [n.d.]. YouTube for Press. Retrieved October 24, 2026 from https://blog.youtube/press/

[19] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik P. A. Lensch. 2023. GGNN: Graph-Based GPU Nearest Neighbor Search. *IEEE Transactions on Big Data (BigData)* 9, 1 (2023), 267–279. https://doi.org/10.1109/TBDATA.2022.3161156

[20] Hao Guo and Youyou Lu. 2025. Achieving Low-Latency Graph-Based Vector Search via Aligning Best-First Search Algorithm with SSD. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 171–186. Retrieved October 24, 2026 from https://www.usenix.org/conference/osdi25/presentation/guo

[21] Hao Guo and Youyou Lu. 2026. OdinANN: Direct Insert for Consistently Stable Performance in Billion-Scale Graph-Based Vector Search. In *Proceedings of the 24th USENIX Conference on File and Storage Technologies (FAST)*. 133–147. Retrieved October 24, 2026 from https://www.usenix.org/conference/fast26/presentation/guo

[22] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3548–3561. https://doi.org/10.14778/3554821.3554843

[23] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 3887–3896. Retrieved October 24, 2026 from https://proceedings.mlr.press/v119/guo20h.html

[24] Jiale Han, Austin Cheung, Yubai Wei, Zheng Yu, Xusheng Wang, Bing Zhu, and Yi Yang. 2025. RAG Meets Temporal Graphs: Time-Sensitive Modeling and Retrieval for Evolving Knowledge. *arXiv preprint arXiv:2510.13590* (2025). https://doi.org/10.48550/arXiv.2510.13590

[25] Han-Wen Hu, Wei-Chen Wang, Yuan-Hao Chang, Yung-Chun Lee, Bo-Rong Lin, Huai-Mu Wang, Yen-Po Lin, Yu-Ming Huang, Chong-Ying Lee, Tzu-Hsiang Su, Chih-Chang Hsieh, Chia-Ming Hu, Yi-Ting Lai, Chung-Kuang Chen, Han-Sung Chen, Hsiang-Pang Li, Tei-Wei Kuo, Meng-Fan Chang, Keh-Chung Wang, Chun-Hsiung Hung, and Chih-Yuan Lu. 2022. ICE: An Intelligent Cognition Engine with 3D NAND-based In-Memory Computing for Vector Similarity Search Acceleration. In *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 763–783. https://doi.org/10.1109/MICRO56248.2022.00058

[26] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-based Retrieval in Facebook Search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 2553–2561. https://doi.org/10.1145/3394486.3403305

[27] Zan Huang, Wingyan Chung, Thian-Huat Ong, and Hsinchun Chen. 2002. A Graph-based Recommender System for Digital Library. In *Proceedings of the 2nd ACM/IEEE-CS Joint Conference on Digital Libraries (JCDL)*. 65–73. https://doi.org/10.1145/544220.544231

[28] Shikhar Jaiswal, Ravishankar Krishnaswamy, Ankit Garg, Harsha Vardhan Simhadri, and Sheshansh Agrawal. 2022. OOD-DiskANN: Efficient and Scalable Graph ANNS for Out-of-Distribution Queries. *arXiv preprint arXiv:2211.12850* (2022). https://doi.org/10.48550/arXiv.2211.12850

[29] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC)*. 585–600. Retrieved October 24, 2026 from https://www.usenix.org/conference/atc23/presentation/jang

[30] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011), 117–128. https://doi.org/10.1109/TPAMI.2010.57

[31] Jinyoung Kim, Dayoon Ko, and Gunhee Kim. 2024. DynamicER: Resolving Emerging Mentions to Dynamic Entities for RAG. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 13752–13770. https://doi.org/10.18653/v1/2024.emnlp-main.762

[32] Ji-Hoon Kim, Yeo-Reum Park, Jaeyoung Do, Soo-Young Ji, and Joo-Young Kim. 2023. Accelerating Large-Scale Graph-Based Nearest Neighbor Search on a Computational Storage Platform. *IEEE Transactions on Computers (TC)* 72, 1 (2023), 278–290. https://doi.org/10.1109/TC.2022.3155956

[33] Sukjin Kim, Seongyeon Park, Si Ung Noh, Junguk Hong, Taehee Kwon, Hunseong Lim, and Jinho Lee. 2025. PathWeaver: A High-Throughput Multi-GPU System for Graph-Based Approximate Nearest Neighbor Search. In *Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC)*. 1501–1517. Retrieved October 24, 2026 from https://www.usenix.org/conference/atc25/presentation/kim

[34] Stanford Vision Lab. [n.d.]. ImageNet. Retrieved October 24, 2026 from https://www.image-net.org/index.php

[35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS)*. 9459–9474. Retrieved October 24, 2026 from https://proceedings.neurips.cc/paper_files/paper/2020/

hash/6b493230205f780e1bc26945df7481e5-Abstract.html

[36] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform. In *Proceedings of the 19th International Middleware Conference Industry (Middleware)*. 9–16. https://doi.org/10.1145/3284028.3284030

[37] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data — Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 32, 8 (2020), 1475–1488. https://doi.org/10.1109/TKDE.2019.2909204

[38] Jiahao Lou, Shufeng Gong, Quan Yu, Hao Guo, Youyou Lu, Song Yu, Yanfeng Zhang, Tiezheng Nie, and Ge Yu. 2025. DGAI: Decoupled On-Disk Graph-Based ANN Index for Efficient Updates and Queries. *arXiv preprint arXiv:2510.25401* (2025). https://doi.org/10.48550/arXiv.2510.25401

[39] Anton Lozhkov, Loubna Ben Allal, Leandro von Werra, and Thomas Wolf. 2024. FineWeb-Edu: the Finest Collection of Educational Content. Retrieved October 24, 2026 from https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu

[40] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020), 824–836. https://doi.org/10.1109/TPAMI.2018.2889473

[41] Microsoft. 2026. DiskANN3: A Composable Vector Indexing Library. Retrieved October 24, 2026 from https://github.com/microsoft/DiskANN

[42] Jason Mohoney, Anil Pacaci, Shihabur Rahman Chowdhury, Umar Farooq Minhas, Jeffery Pound, Cedric Renggli, Nima Reyhani, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2024. Incremental IVF Index Maintenance for Streaming Vector Search. *arXiv preprint arXiv:2411.00970* (2024). https://doi.org/10.48550/arXiv.2411.00970

[43] Jason Mohoney, Devesh Sarda, Mengze Tang, Shihabur Rahman Chowdhury, Anil Pacaci, Ihab F. Ilyas, Theodoros Rekatsinas, and Shivaram Venkataraman. 2025. Quake: Adaptive Indexing for Vector Search. In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 153–169. Retrieved October 24, 2026 from https://www.usenix.org/conference/osdi25/presentation/mohoney

[44] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 4236–4247. https://doi.org/10.1109/ICDE60146.2024.00323

[45] OpenAI. 2024. New embedding models and API updates. Retrieved October 24, 2026 from https://openai.com/index/new-embedding-models-and-api-updates/

[46] OpenAI. 2024. Vector Embeddings – OpenAI API. Retrieved October 24, 2026 from https://developers.openai.com/api/docs/guides/embeddings

[47] Yu Pan, Jianxin Sun, and Hongfeng Yu. 2023. LM-DiskANN: Low Memory Footprint in Disk-Native Dynamic Graph-Based ANN Indexing. In *Proceedings of the 2023 IEEE International Conference on Big Data (BigData)*. 5987–5996. https://doi.org/10.1109/BigData59044.2023.10386517

[48] Qdrant. 2025. Qdrant: High-Performance Vector Search at Scale. Retrieved October 24, 2026 from https://qdrant.tech/

[49] Ziyue Qiu, Juncheng Yang, Juncheng Zhang, Cheng Li, Xiaosong Ma, Qi Chen, Mao Yang, and Yinlong Xu. 2023. FrozenHot Cache: Rethinking Cache Management for Modern Hardware. In *Proceedings of the 18th European Conference on Computer Systems (EuroSys)*. 557–573. https://doi.org/10.1145/3552326.3587446

[50] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*. 8748–8763. Retrieved October 24, 2026 from https://proceedings.mlr.press/v139/radford21a.html

[51] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Proceedings of the Advances in Neural Information Processing Systems 33 (NeurIPS)*. 10672–10684. Retrieved October 24, 2026 from https://proceedings.neurips.cc/paper_files/paper/2020/hash/788d986905533aba051261497ecffcbb-Abstract.html

[52] Yuanming Ren, Juncheng Zhang, Yanjing Ren, Rui Yang, Di Wu, and Patrick P. C. Lee. 2026. Decoupling Vector Data and Index Storage for Space Efficiency. *arXiv preprint arXiv:2604.09173* (2026). https://doi.org/10.48550/arXiv.2604.09173

[53] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. *arXiv preprint arXiv:2105.09613* (2021). https://doi.org/10.48550/arXiv.2105.09613

[54] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node. In *Proceedings of the Advances in Neural Information Processing Systems 32 (NeurIPS)*. Retrieved October 24, 2026 from https://proceedings.neurips.cc/paper_files/paper/2019/hash/09853c7fb1d3f8ee67a61b6bf4a7f8e6-Abstract.html

[55] Philip Sun, David Simcha, Dave Dopson, Ruiqi Guo, and Sanjiv Kumar. 2023. SOAR: Improved Indexing for Approximate Nearest Neighbor Search. In *Proceedings of the Advances in Neural Information Processing Systems 36 (NeurIPS)*. 3189–3204. https://doi.org/10.52202/075280-0141

[56] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*. 1135–1150. Retrieved October 24, 2026 from https://www.usenix.org/conference/atc24/presentation/tian

[57] Karthik Venkatasubba, Saim Khan, Somesh Singh, Harsha Vardhan Simhadri, and Jyothi Vedurada. 2025. BANG: Billion-Scale Approximate Nearest Neighbour Search Using a Single GPU. *IEEE Transactions on Big Data (BigData)* 11, 6 (2025), 3142–3157. https://doi.org/10.1109/TBDATA.2025.3581085

[58] Henrique Schechter Vera, Sahil Dua, Biao Zhang, Daniel Salz, Ryan Mullins, Sindhu Raghuram Panyam, Sara Smoot, Iftekhar Naim, Joe Zou, Feiyang Chen, Daniel Cer, Alice Lisak, Min Choi, Lucas Gonzalez, Omar Sanseviero, Glenn Cameron, Ian Ballantyne, Kat Black, Kaifeng Chen, Weiyi Wang, Zhe Li, Gus Martins, Jinhyuk Lee, Mark Sherwood, Juyeong Ji, Renjie Wu, Jingxiao Zheng, Jyotinder Singh, Abheesht Sharma, Divyashree Sreepathihalli, Aashi Jain, Adham Elarabawy, AJ Co, Andreas Doumanoglou, Babak Samari, Ben Hora, Brian Potetz, Dahun Kim, Enrique Alfonseca, Fedor Moiseev, Feng Han, Frank Palma Gomez, Gustavo Hernández Ábrego, Hesen Zhang, Hui Hui, Jay Han, Karan Gill, Ke Chen, Koert Chen, Madhuri Shanbhogue, Michael Boratko, Paul Suganthan, Sai Meher Karthik Duddu, Sandeep Mariserla, Setareh Ariafar, Shanfeng Zhang, Shijie Zhang, Simon Baumgartner, Sonam Goenka, Steve Qiu, Tanmaya Dabral, Trevor Walker, Vikram Rao, Waleed Khawaja, Wenlei Zhou, Xiaoqi Ren, Ye Xia, Yichang Chen, Yi-Ting Chen, Zhe Dong, Zhongli Ding, Francesco Visin, Gaël Liu, Jiageng Zhang, Kathleen Kenealy, Michelle Casbon, Ravin Kumar, Thomas Mesnard, Zach Gleicher, Cormac Brick, Olivier Lacombe, Adam Roberts, Qin Yin, Yunhsuan Sung, Raphael Hoffmann, Tris Warkentin, Armand Joulin, Tom Duerig, and Mojtaba Seyedhosseini. 2025. EmbeddingGemma: Powerful and Lightweight Text Representations. *arXiv preprint arXiv:2509.20354* (2025). https://doi.org/10.48550/arXiv.2509.20354

[59] Vespa. 2026. Vespa. Retrieved October 24, 2026 from https://github.com/vespa-engine/vespa

[60] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 2614–2627. https://doi.org/10.1145/3448016.3457550

[61] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 1 (2024), 1–27. https://doi.org/10.1145/3639269

[62] Weaviate. 2026. Weaviate. Retrieved October 24, 2026 from https://github.com/weaviate/weaviate

[63] Haike Xu, Magdalen Dobson Manohar, Philip A. Bernstein, Badrish Chandramouli, Richard Wen, and Harsha Vardhan Simhadri. 2025. In-Place Updates of a Graph Index for Streaming Approximate Nearest Neighbor Search. *arXiv preprint arXiv:2502.13826* (2025). https://doi.org/10.48550/arXiv.2502.13826

[64] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025. A-Mem: Agentic Memory for LLM Agents. In *Proceedings of the Advances in Neural Information Processing Systems 38 (NeurIPS)*. 20004–20031. https://doi.org/10.52202/085713-0593

[65] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. 545–561. https://doi.org/10.1145/3600006.3613166

[66] Peiqi Yin, Xiao Yan, Qihui Zhou, Hui Li, Xiaolu Li, Meiling Wang, Lin Zhang, Xin Yao, and James Cheng. 2026. GPS: Revisiting the Data Layout for Disk-based High-Dimensional Vector Search. *Proceedings of the ACM on Management of Data (PACMMOD)* 4, 3 (2026), 1–29. https://doi.org/10.1145/3802069

[67] Song Yu, Shengyuan Lin, Shufeng Gong, Yongqing Xie, Ruicheng Liu, Yijie Zhou, Ji Sun, Yanfeng Zhang, Guoliang Li, and Ge Yu. 2025. A Topology-Aware Localized Update Strategy for Graph-Based ANN Index. *Proceedings of the VLDB Endowment* 19, 3 (2025), 495–508. https://doi.org/10.14778/3778092.3778108

[68] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated Proximity Graph Approximate Nearest Neighbor Search and Construction. In *Proceedings of the 2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 552–564. https://doi.org/10.1109/ICDE53745.2022.00046

[69] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, and Yu Wang. 2023. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 283–296. https://doi.org/10.1145/3613424.3614292

[70] Minjia Zhang, Wenhan Wang, and Yuxiong He. 2022. GraSP: Optimizing Graph-based Nearest Neighbor Search with Subgraph Sampling and Pruning. In *Proceedings of the 15th ACM International Conference on Web Search and Data Mining (WSDM)*. 1395–1405. https://doi.org/10.1145/3488560.3498425

[71] Ziyu Zhang, Yuanhao Wei, Joshua Engels, and Julian Shun. 2026. CleanANN: Efficient and Robust Full Dynamism in Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 38th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*. 247–260. https://doi.org/10.1145/3816782.3819219

[72] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *Proceedings of the 2020 IEEE 36th International Conference on Data Engineering (ICDE)*. 1033–1044. https://doi.org/10.1109/ICDE48307.2020.00094

[73] Lei Zhu, Chaoqun Zheng, Weili Guan, Jingjing Li, Yang Yang, and Heng Tao Shen. 2024. Multi-Modal Hashing for Efficient Multimedia Retrieval: A Survey. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 36, 1 (2024), 239–260. https://doi.org/10.1109/TKDE.2023.3282921

[74] Zhenhua Zhu, Jun Liu, Guohao Dai, Shulin Zeng, Bing Li, Huazhong Yang, and Yu Wang. 2023. Processing-In-Hierarchical-Memory Architecture for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 60th ACM/IEEE Design Automation Conference (DAC)*. 1–6. https://doi.org/10.1109/DAC56929.2023.10247946